PRACTICAL 2 CODE TANTRA

Name: Satyam B Sonawane

PRN: 202501040100

Division: CS1

Batch: C15

2.1.1. List operations

Write a Python program that implements a menu-driven interface for managing a list of integers. The program should have the following menu options:
1. Add
2. Remove
3. Display
4. Quit

The program should repeatedly prompt the user to enter a choice from the menu. Depending on the choice selected, the program should perform the following actions:

- **Add**: Prompts the user to enter an integer and add it to the integer list. If the input is not a valid integer, display "Invalid input".
- **Remove:** Prompts the user to enter an integer to remove from the list. If the integer is found in the list, remove it; otherwise, display "Element not found". If the list is empty, display "List is empty".
- **Display**: Displays the current list of integers. If the list is empty, display "List is empty".
- **Quit**: Exits the program.
- The program should handle invalid menu choices by displaying "Invalid choice".Ensure that the program continues to prompt the user until they choose to quit (option 4).

Code:

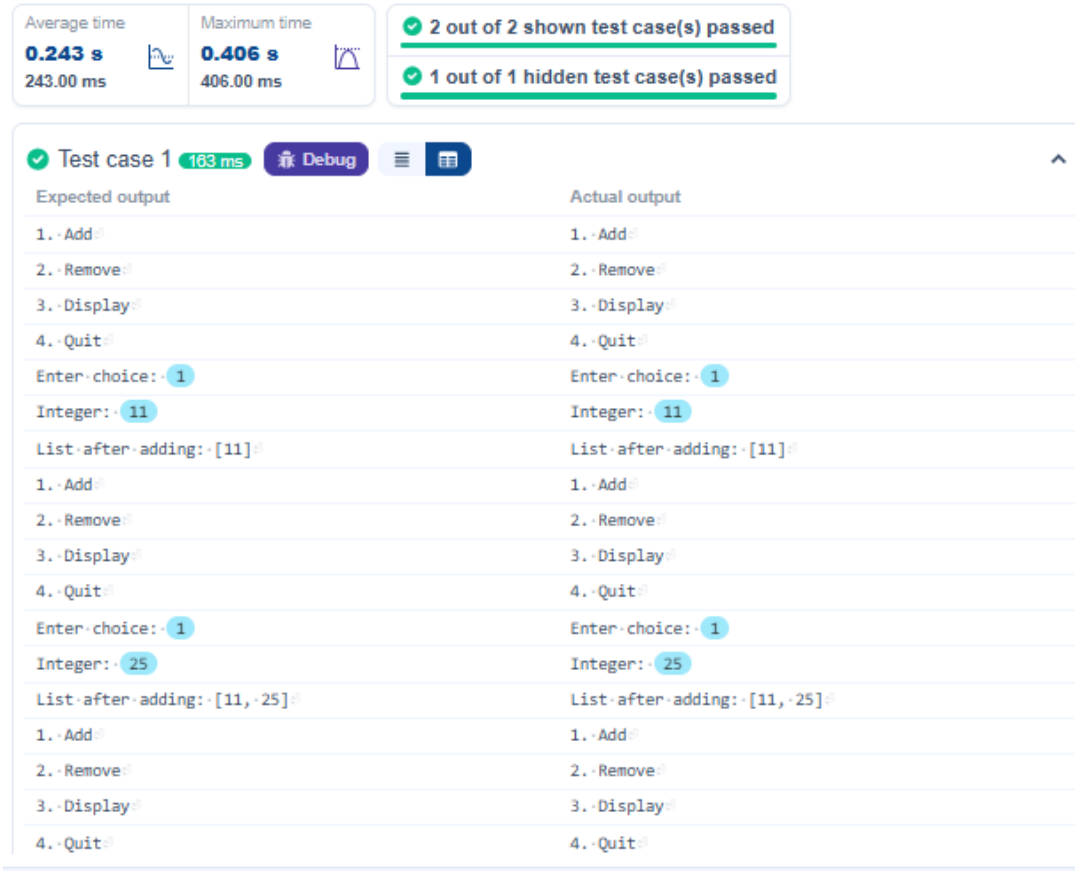
```
def add_element(lst):
    try:
        num = int(input("Integer: "))
        lst.append(num)
        print("List after adding:", lst)
    except ValueError:
        print("Invalid input")
```

```
def remove_element(lst):
    if len(lst) == 0:
        print("List is empty")
        return
    try:
        num = int(input("Integer: "))
        if num in lst:
            lst.remove(num)
            print("List after removing:", lst)
        else:
            print("Element not found")
    except ValueError:
        print("Invalid input")
def display_list(lst):
    if len(lst) == 0:
        print("List is empty")
    else:
        print(lst)
def main():
    lst = list()
    while True:
        print("1. Add")
        print("2. Remove")
        print("3. Display")
        print("4. Quit")
        choice = input("Enter choice: ")

        if choice == '1':
```

```
            add_element(lst)
        elif choice == '2':
            remove_element(lst)

        elif choice == '3':
            display_list(lst)
        elif choice == '4':
            break
        else:
            print("Invalid choice")
if __name__ == "__main__":
    main()
```



```
Enter choice: 2
Integer: 26
Element not found
1. Add
2. Remove
3. Display
4. Quit
Enter choice: 2
Integer: 11
List after removing: [25]
1. Add
2. Remove
3. Display
4. Quit
Enter choice: 2
Integer: 25
List after removing: []
1. Add
2. Remove
3. Display
4. Quit
```

```
Enter choice: 2
Integer: 26
Element not found
1. Add
2. Remove
3. Display
4. Quit
Enter choice: 2
Integer: 11
List after removing: [25]
1. Add
2. Remove
3. Display
4. Quit
Enter choice: 2
Integer: 25
List after removing: []
1. Add
2. Remove
3. Display
4. Quit
```

```
Enter choice: 3
List is empty
1. Add
2. Remove
3. Display
4. Quit
Enter choice: 2
List is empty
1. Add
2. Remove
3. Display
4. Quit
Enter choice: 8
Invalid choice
1. Add
2. Remove
3. Display
4. Quit
Enter choice: 4
```

```
Enter choice: 3
List is empty
1. Add
2. Remove
3. Display
4. Quit
Enter choice: 2
List is empty
1. Add
2. Remove
3. Display
4. Quit
Enter choice: 8
Invalid choice
1. Add
2. Remove
3. Display
4. Quit
Enter choice: 4
```

2.1.2. Dictionary Operations

Write a Python program to perform insertion, update, deletion, and traversal operations on a dictionary. An initial dictionary containing 10 predefined records is already given in the program.

Operations to be Performed:

1. Insertion – Insert a new key-value pair into the dictionary using user input.
2. Update – Update the value of an existing key using user input.
3. Deletion – Delete a specified key from the dictionary using user input.
4. Traversal – Traverse the final dictionary and display all key-value pairs.

Input and Output Format:

1. Read an integer representing the key to be inserted and a string representing its value. Insert this new key-value pair into the dictionary and display the dictionary after insertion.
2. Read an integer representing the key to be updated and a string representing the new value. Update the value of the specified key (only if it exists) and display the dictionary after the update.
3. Read an integer representing the key to be deleted. Delete the specified key from the dictionary (only if it exists) and display the dictionary after deletion.
4. Finally, traverse the dictionary and display all key-value pairs.

The program should also display the original dictionary before performing any operations. Each output should be printed with appropriate messages.

Note:

- All operations must be performed using dictionary methods.
- Perform deletion only if possible; leave the dictionary unchanged.
- Refer to the visible test cases for better understanding and strictly match with the input/outputs.

Code:

```
# Initial dictionary with 10 predefined records
student = {
    1: "Amit",
    2: "Riya",
    3: "Kiran",
    4: "Neha",
```

```
    5: "Arjun",
    6: "Pooja",
    7: "Rahul",
    8: "Sneha",
    9: "Vikram",
    10: "Anjali"
}

# write your code here...
d = {1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun',
     6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali'}

# Display original dictionary
print("Original Dictionary:", d)

# Insertion
key_insert = int(input())
value_insert = input()
d[key_insert] = value_insert
print("After Insertion:", d)

# Update
key_update = int(input())
value_update = input()
if key_update in d:
    d[key_update] = value_update
print("After Update:", d)
```

```
# Deletion
key_delete = int(input())
if key_delete in d:
    del d[key_delete]
print("After Deletion:", d)

# Traversal
print("Traversing Dictionary:")
for key, value in d.items():
    print(key, ":", value)
```

2.2.1. Linear search Technique

Write a program to check whether the given element is present or not in the array of elements using linear search.

Input format:

- The first line of input contains the array of integers which are separated by space
- The last line of input contains the key element to be searched

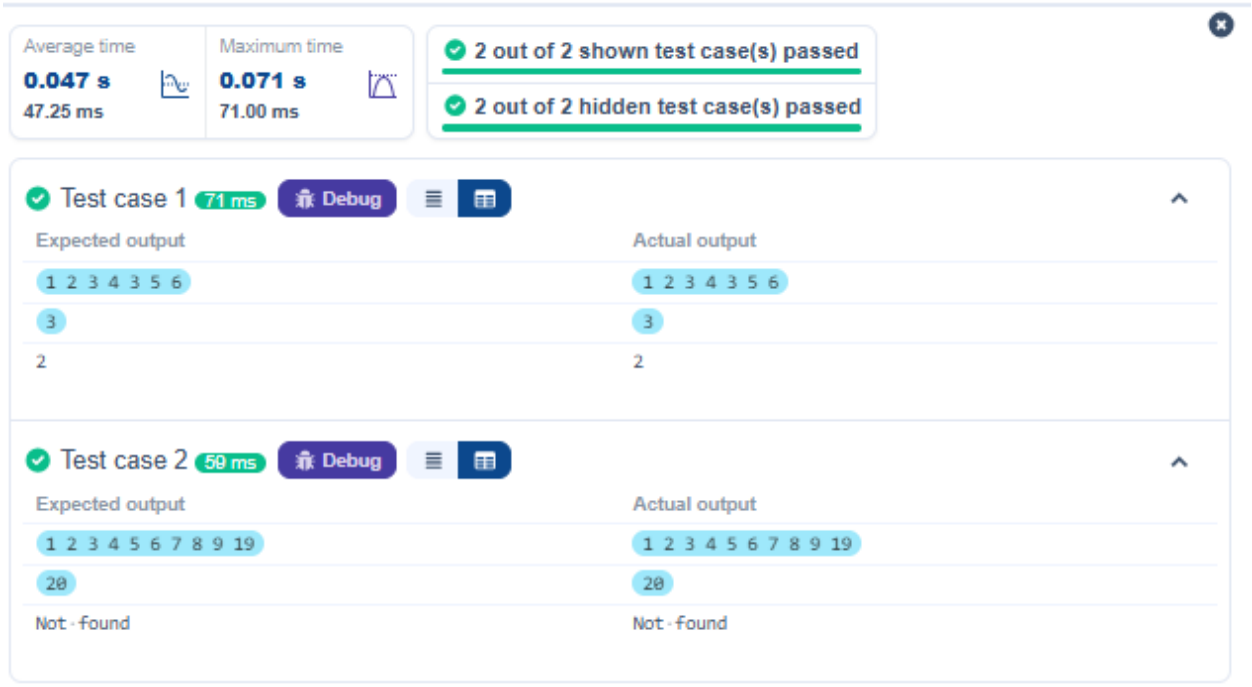
Output format:

- If the element is found, print the index. If the element found multiple times, print the starting index
- If the element is not found, print **Not found**.

Sample Test Case:
Input:
1 2 3 4 3 5 6
3
Output:
2

Code:

```
def linear_search(arr, key):
        for i in range(len(arr)):
                if arr[i] == key:
                        return i
        return "Not found"
arr = list(map(int, input().split()))
key = int(input())
result = linear_search(arr, key)
print(result)
```



2.2.2. Captain of the Team

You are provided with the heights of 11 cricket players (in centimeters). Your task is to identify the tallest player, who will be selected as the captain of the team.

Input Format:
The first line of input will contain 11 integers, each representing the height of a player (in centimeters), each separated by a space.

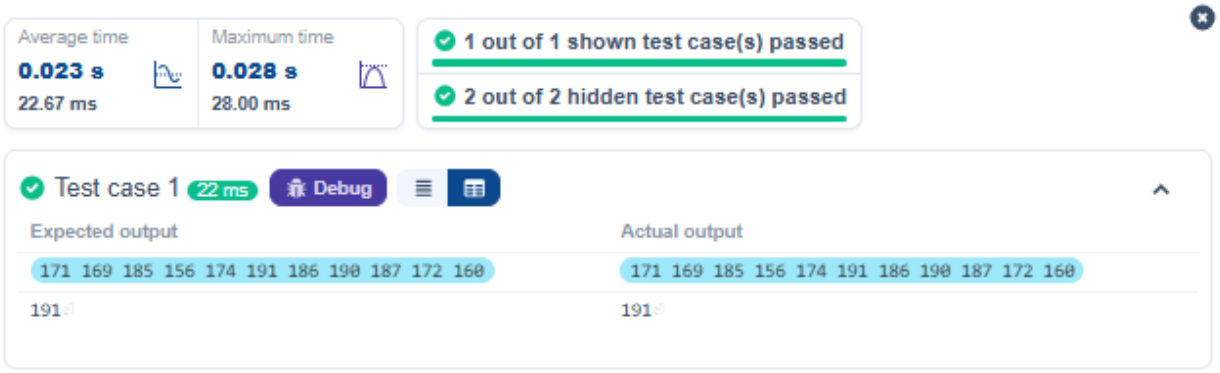
Output Format
The output should be the height (in centimeters) of the tallest player.

Code:

```
heights = list(map(int, input().split()))

tallest = max(heights)

print(tallest)
```


Average time
0.023 s
22.67 ms
Maximum time
0.028 s
28.00 ms
1 out of 1 shown test case(s) passed
2 out of 2 hidden test case(s) passed
Test case 1
22 ms
Debug
Expected output
171 169 185 156 174 191 186 190 187 172 160
191
Actual output
171 169 185 156 174 191 186 190 187 172 160
191